## Exercise 2: Implementing Dependency Injection

### Scenario:

In the library management application, you need to manage the dependencies between the BookService and BookRepository classes using Spring's IoC and DI.

## SOLUTION:

### applicationContext.xml:

```xml
<?xml version="1.0" encoding="UTF-8"?>

<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
       http://www.springframework.org/schema/beans
       https://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="bookRepository"
          class="com.library.repository.BookRepository"/>

    <bean id="bookService"
          class="com.library.service.BookService">

        <property name="bookRepository"
                  ref="bookRepository"/>

    </bean>

</beans>
```

### BookRepository.java:

```java
package com.library.repository;

public class BookRepository {

    public String getBook() {
        return "Spring Framework";
    }
}
```

### BookService.java:

```java
package com.library.service;

import com.library.repository.BookRepository;

public class BookService {

    private BookRepository bookRepository;

    public void setBookRepository(BookRepository bookRepository) {
        this.bookRepository = bookRepository;
    }

    public void displayBook() {
        System.out.println("Book Name: " + bookRepository.getBook());
    }
}
```

**MainApp.java:**

```java
package com.library;

import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

import com.library.service.BookService;

public class MainApp {

    public static void main(String[] args) {

        ApplicationContext context =
                new ClassPathXmlApplicationContext("applicationContext.xml");

        BookService service =
                context.getBean("bookService", BookService.class);

        service.displayBook();
    }
}
```

## OUTPUT: